

ProAFL: Process Metrics-Based Annotation Fault Localization for Object Detection Systems

Abstract—Object detection models are widely used in intelligent software systems, and their reliability depends heavily on the quality of training annotations. However, object detection datasets often contain annotation faults, and thoroughly inspecting them is expensive. Therefore, annotation fault localization is urgently needed to reduce inspection costs. Since faulty annotations may exhibit abnormal model responses during training, process metrics across training epochs provide a promising perspective for annotation fault localization. However, existing methods often overlook this process perspective.

We propose ProAFL, a process metrics-based method for localizing annotation faults in object detection datasets. ProAFL collects training process metrics and constructs process features. Based on these features, ProAFL ranks suspicious annotations to reduce manual inspection costs under limited budgets. Given a fixed inspection budget, manual reviewers correct the inspected annotations, and we retrain the model on the corrected dataset to improve performance. To evaluate ProAFL, we conduct experiments on three datasets and three representative object detection models, and compare it with six baseline methods. The results show that ProAFL outperforms baselines in ranking indicators. Moreover, correcting annotations prioritized by ProAFL and retraining the model improves detection performance. These results demonstrate the effectiveness of training process metrics for annotation fault localization.

Index Terms—process metrics, annotation fault localization, object detection

I. INTRODUCTION

Object detection is a fundamental computer vision task that detects objects and localizes them with bounding boxes in visual scenes [1]. Object detection systems serve as a foundation for many intelligent applications, such as autonomous driving, traffic-scene analysis, and unmanned aerial vehicle perception [2]–[4]. With the advances in deep learning, the development of modern object detection systems has increasingly shifted toward a data-driven paradigm [5]. Under this paradigm, system performance depends not only on model architectures and training algorithms, but also heavily on the quality of training data [6]. Large-scale annotated datasets have therefore become a key driver of progress in object detection, enabling deep models to learn richer visual representations and achieve improvements in performance [7]. However, data faults are difficult to avoid during data collection and manual annotation. Such low-quality data can eventually lead to undesirable consequences, such as degraded model performance and robustness [8], fairness [9], and reliability [10].

Among data faults in object detection datasets, annotation faults are especially critical because they directly define the supervision signals used for model training. Such faults are particularly likely to occur because the annotation task is more

complex than image classification [11]. Instead of assigning one label to an entire image, annotators need to label each object instance with both a category and a bounding box, which requires finer-grained judgment and more precise spatial localization [12]. This increased annotation complexity makes faults more likely and harder to inspect, as they may involve incorrect category labels, inaccurate bounding boxes, or missing objects. Since a single image may contain many object annotations, manual inspection requires considerable effort.

To reduce the cost of finding data faults, recent studies have explored data fault localization from different perspectives. Data Shapley estimates the contribution of each training sample to model performance and helps identify abnormal samples [13]. Li et al. localized suspicious data features that lead to unfair model behavior [14], and DFauLo ranked faulty training data using model mutants and differential features [15]. More closely related to object detection, ObjectLab evaluates image-level annotation quality using trained models [16], while Dactive decomposes object detection data into foreground and background classification samples to localize faulty annotations [17]. These studies demonstrate that suspicious data can be prioritized for inspection. However, they rely on static evidence before or after training, overlooking fault signals that emerge during training.

To efficiently capture information during training, we propose a novel approach that takes a different angle: inspired by process metrics [18] in traditional software engineering, we treat the training procedure of deep learning models as the iterative development process of software. Process metrics characterize software evolution using historical information such as code changes, development activities, and past defects, and have been shown to be effective in revealing defect risks [19]–[23]. Similarly, by continuously collecting relevant quality factors throughout the modeling training process and ultimately integrating them in line with the paradigm of process metrics, we argue that it would effectively identify annotation faults. Specifically, because the model training process is naturally divided into multiple epochs, the predictions generated during each epoch can reflect how the model responds to each annotation. In our methods, we aim to extract two types of prediction information: confidence (the model’s classification certainty) [24] and Intersection over Union (IoU, the localization consistency between a prediction and an annotation) [25] and integrate the sequence of confidence and IoU extracted from training processes to construct process metrics to describe the suspiciousness of Annotation faults.

In this paper, we propose ProAFL, a **P**rocess Metrics-based

Annotation Fault Localization method for object detection datasets. ProAFL follows three steps. First, it collects prediction metrics across training epochs and constructs process features. For explicit annotation faults, i.e., faults associated with existing annotation boxes, ProAFL constructs box-level process features. For implicit annotation faults, i.e., missing annotations without existing boxes, ProAFL constructs image-level process features. Second, ProAFL comprehensively evaluates multi-dimensional process features at both levels, assigns suspiciousness scores, and produces a unified priority ranking to reduce manual inspection cost. Third, under a limited inspection budget, human inspectors correct suspicious annotations according to the ranking, and the model is retrained on the corrected dataset to improve performance.

We evaluate ProAFL on three datasets from different application scenarios, including VOC [26], KITTI [27], and VisDrone [28], using three representative object detection models: YOLOv7 [29], Faster R-CNN [30], and RT-DETR [31]. Across nine dataset-model settings, we compare ProAFL with six baseline methods: Dactive (ISSTA'24) [17], Loss (TNNLS'23) [32], DeepGini (ISSTA'20) [33], Entropy (BMVC'18) [34], Margin [35] and ObjectLab [16]. The experimental results show that ProAFL achieves better annotation fault localization performance than the baselines. Moreover, retraining the model on the corrected data after manually correcting suspicious annotations prioritized by ProAFL yields better detection performance.

The main contributions of this paper are as follows:

- **Method.** We propose ProAFL, the first process metrics-based method for localizing annotation faults in object detection datasets. It uses process features collected across training epochs to rank annotations for inspection.
- **Study.** Through experiments on three object detection datasets and three models, we find that training process metrics are effective in both localizing faulty annotations and improving model performance afterward.

II. BACKGROUND AND MOTIVATION

A. Preliminary

1) *Object Detection*: Object detection is a fundamental computer vision task that aims to identify object instances in an image and localize them with bounding boxes. It plays an important role in many scenarios, including autonomous driving, UAV perception, and traffic scene analysis [2]–[4]. In deep learning, mainstream object detection models can be broadly divided into three categories. Two-stage models, such as Faster R-CNN [30], first generate candidate object proposals and then classify and refine them. One-stage models, represented by the YOLO series [29], directly predict object categories and bounding boxes from dense image locations for higher inference efficiency. Transformer-based models, such as RT-DETR [31], formulate detection as a set prediction problem and use attention mechanisms to model global image context. The training of these models relies on object detection datasets, where each image is annotated with a set of ground-

truth boxes. Each box specifies an object's location, usually by box coordinates, together with its category label [12].

2) *Annotation Faults*: Compared with image classification labels, object detection annotations are more complex because they are object-level and contain both category labels and bounding-box locations. Following Dactive [17], we consider four types of annotation faults: class faults, localization faults, redundant faults, and missing faults, corresponding to incorrect categories, inaccurate boxes, extra boxes, and unannotated objects, respectively. Since class, localization, and redundant faults are associated with existing boxes, we treat them as explicit annotation faults. In contrast, missing faults have no corresponding ground-truth boxes, so we treat them as implicit annotation faults. Recent studies have begun to debug annotation faults in object detection datasets by ranking suspicious images or annotations for inspection. For example, ObjectLab [16] uses a trained detector to estimate image-level annotation quality and prioritize images for review. Dactive [17] decomposes object detection data into foreground and background classification samples and applies robust learning to localize both faulty annotations and missing objects.

B. Motivation

To illustrate the main idea of our method, we first introduce process metrics from traditional software engineering that inspired it. Unlike static metrics that capture a single snapshot of software, process metrics leverage historical information, such as code changes, activities, and past defects, to reflect software evolution. Such evolution information has been shown to predict defect density, identify risky entities, and outperform static indicators from a single snapshot [18]–[23].

Inspired by process metrics in traditional software engineering, we regard information generated during model training as process information that characterizes the development of deep learning software. In object detection, a model outputs predicted boxes, each containing box coordinates and a confidence score [29]–[31]. **Confidence** reflects the model's certainty about the predicted class, while **Intersection over Union (IoU)** measures the spatial overlap between a predicted box and an annotation, and is commonly used for prediction-annotation matching and localization evaluation [27], [28]. Therefore, for each annotation, we collect the confidence and IoU produced across training epochs and regard the resulting confidence and IoU sequences as its process metrics. By aggregating these sequences, we construct process features to distinguish faulty annotations from correct ones.

To preliminarily validate this idea, we conduct a pilot experiment using YOLOv7 on the VOC dataset. During training over 50 epochs, we collect predicted boxes matched to each annotation (i.e., each ground truth box) at each epoch and record the confidence of the matched prediction and its IoU with the corresponding annotation. Based on the resulting confidence and IoU sequences, we construct two intuitive process features by aggregation: **Confidence Mean (CM)** and **IoU Mean (IM)**, which are the average confidence and average IoU over all 50 epochs, respectively, as detailed in

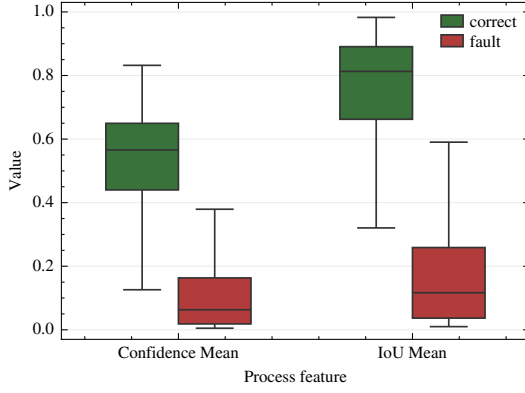


Fig. 1: Distributions of process features for correct and faulty annotations on VOC using YOLOv7.

Section III-A. We then compare the distributions of these two features between correct and faulty annotations. As shown in Fig. 1, correct annotations generally have higher CM and IM values than faulty annotations, indicating that training process features can provide useful signals for distinguishing faulty annotations from correct ones. These observations suggest the potential of using process metrics for diagnosing annotation faults. In the next section, we present two groups of process features derived from training-process metrics to localize faulty annotations in the training set.

III. METHODOLOGY

This section details our approach, as shown in Fig. 2, which consists of three steps: Process Feature Construction, Suspiciousness score and Ranking, and Correction and Retraining.

A. Process Feature Construction

Training Information Collection: Given a noisy training set with faulty annotations, ProAFL first trains an object detection model for T epochs and performs inference on the training set at the end of each epoch. In this way, ProAFL collects prediction boxes throughout the training process. Based on these predictions, ProAFL further obtains process metrics and constructs process features at two granularities: box-level features for explicit annotation faults and image-level features for missing annotation faults.

Box-level feature construction. For explicit annotation faults (i.e., class faults, localization faults, and redundant boxes), corresponding GT boxes exist in the training set. Therefore, ProAFL collects box-level process data by matching prediction boxes to GT boxes. Specifically, it first groups the GT and prediction boxes from the same image by their annotated and predicted classes, respectively. Within each class group, prediction boxes are processed in descending order of confidence. For each prediction box, ProAFL selects the unmatched GT box with the highest IoU. A match is established if their IoU exceeds a threshold; otherwise, the prediction box is considered unmatched. ProAFL enforces one-to-one matching in each epoch so that each prediction box can be matched to

at most one GT box, and each GT box can be matched to at most one prediction box. After T epochs, ProAFL obtains a sequence of matched prediction boxes for each GT box and extracts the confidence and IoU with the GT box, forming a **Confidence sequence** and an **IoU sequence**. These sequences constitute box-level process metrics.

Based on the box-level process metrics, ProAFL constructs eight box-level process features for localizing explicit annotation faults: (1) Confidence Mean (CM) and (2) IoU Mean (IM), which measure the average confidence and localization quality over the entire training process; (3) Early Confidence Mean (ECM) and (4) Early IoU Mean (EIM), which characterize confidence and localization quality during the first w epochs; (5) Late Confidence Mean (LCM) and (6) Late IoU Mean (LIM), which characterize these properties during the last w epochs, respectively; and (7) Delay Confidence (DC) and (8) Delay IoU (DI), which measure learning delay as the normalized first epoch at which confidence or IoU reaches half of its corresponding late stage mean, respectively. Table I summarizes these features and their definitions.

To further illustrate box-level feature construction, Fig. 3 shows a concrete example from our experiment. Fig. 3a presents six training snapshots of one annotated object, where the green box is the GT box and the blue boxes are matched predictions at selected epochs (1, 5, 10, 40, 45, and 50). Fig. 3b shows the IoU sequence of this GT box over all 50 epochs. We divide this sequence into three stages: all 50 epochs (All), the first 10 epochs (Early), and the last 10 epochs (Late). As shown in Fig. 3c, the mean IoUs over these stages are used to compute IM, EIM, and LIM, with values of 0.919, 0.812, and 0.950, respectively. DI captures localization delay by finding the first epoch whose IoU reaches half of LIM; in this example, the threshold is reached at epoch 1 ($0.561 > 0.475$, where $0.475 = 0.950/2$), yielding $DI = 1/50$.

Image-level feature construction. For missing annotation faults, no corresponding GT boxes exist in the training set. Thus, ProAFL cannot collect box-level process data through the above matching rules. Instead, it retains high-confidence predictions generated during the late training stage that are not matched to any GT box and organizes them by image. For each image, ProAFL uses a union-find algorithm [36] to cluster spatially close predictions according to their pairwise IoU, producing a set of high-confidence unmatched prediction clusters. These clusters constitute image-level process metrics.

Based on the image-level process metrics, ProAFL constructs four image-level process features for localizing missing-annotation faults: (1) Cluster Confidence Mean (CCM), which measures the average confidence of the predictions in the cluster; (2) Cluster IoU Mean (CIM), which measures the spatial overlap among the prediction boxes in the cluster and reflects their spatial compactness; (3) Cluster Class Consistency (CCC), which measures the consistency of predicted classes within the cluster; and (4) Cluster Epoch Span (CES), which measures how persistently the predictions in the cluster appear during training. Intuitively, a cluster with high confidence, compact spatial distribution, high class con-

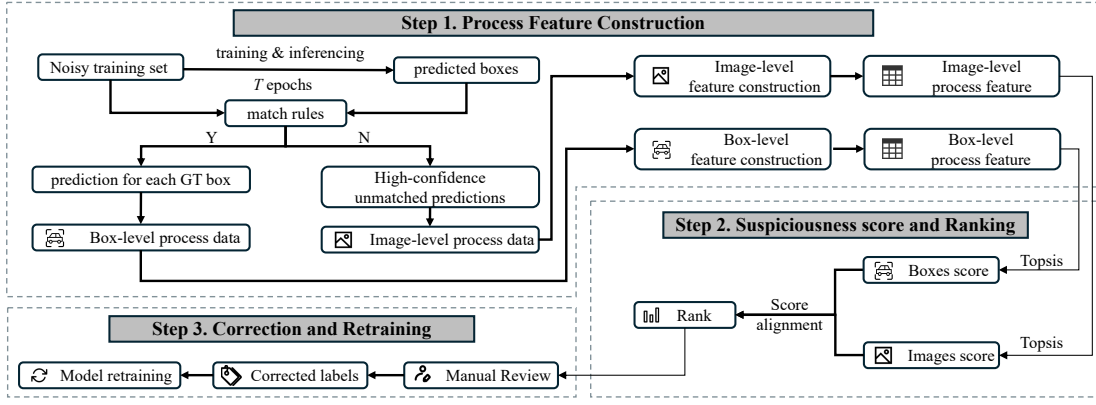


Fig. 2: An overview of ProAFL. Step 1 constructs process features at the box and image levels. Step 2 computes suspiciousness scores and produces a unified ranking. Step 3 guides the correction of annotations and the retraining of the model under a limited inspection budget.

TABLE I: Definitions of the eight box-level process features. $confidence_{list}$ and IoU_{list} denote the confidence and IoU metric lists, respectively. $Epochs$ denotes the total number of training epochs, w denotes the window size covering either the first or last w epochs, e denotes a specific epoch, and $confidence^e$ and IoU^e denote the confidence and IoU recorded at epoch e .

Process feature	Description	Equation
Confidence Mean (CM)	Mean confidence over all epochs.	$CM = \text{Mean}(confidence_{list})$
Early Confidence Mean (ECM)	Mean confidence over the first w epochs.	$ECM = \text{Mean}(confidence_{list}[:w])$
Late Confidence Mean (LCM)	Mean confidence over the last w epochs.	$LCM = \text{Mean}(confidence_{list}[-w:])$
IoU Mean (IM)	Mean IoU over all epochs.	$IM = \text{Mean}(IoU_{list})$
Early IoU Mean (EIM)	Mean IoU over the first w epochs.	$EIM = \text{Mean}(IoU_{list}[:w])$
Late IoU Mean (LIM)	Mean IoU over the last w epochs.	$LIM = \text{Mean}(IoU_{list}[-w:])$
Delay Confidence (DC)	First epoch reaching half the late-stage confidence mean.	$DC = \min\{e \mid confidence^e > 0.5LCM\} / Epochs$
Delay IoU (DI)	First epoch reaching half the late-stage IoU mean.	$DI = \min\{e \mid IoU^e > 0.5LIM\} / Epochs$

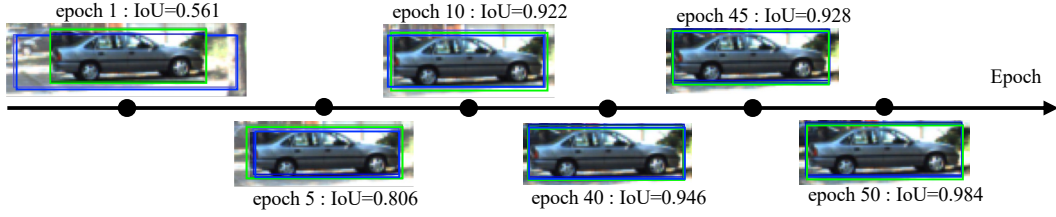
sistency, and persistent occurrence is more likely to correspond to a missing annotation in the image. Table II summarizes these features and their definitions. These features jointly characterize the reliability, location stability, semantic consistency, and temporal persistence of each unmatched prediction cluster.

To further illustrate image-level feature construction, Fig. 4 shows a concrete example from our experiment. The upper part shows an image from the training set. The object on the left side of the image is a missing fault, and the colored boxes over it form a prediction cluster collected from high-confidence predictions that are unmatched with any GT box during the last five epochs (46–50). The object on the right side of the image is correctly annotated. The Cluster table lists detailed information about the predicted boxes in the cluster, including the predicted box ID, epoch ID, predicted class, and confidence. The IoU matrix table reports the pairwise IoUs among the five predicted boxes in the cluster, and the last row reports the mean value of the IoU matrix. The four image-level process features can be directly computed from the two tables. CCM is the mean confidence of the cluster (0.835, i.e., the mean of the confidence column in the cluster table). CIM is the mean value of the IoU matrix (0.979). CCC is the ratio of the dominant class (i.e., car) count to the number of predicted boxes (1.000). CES is the ratio of the number of appeared epochs to the number of collected epochs (1.000).

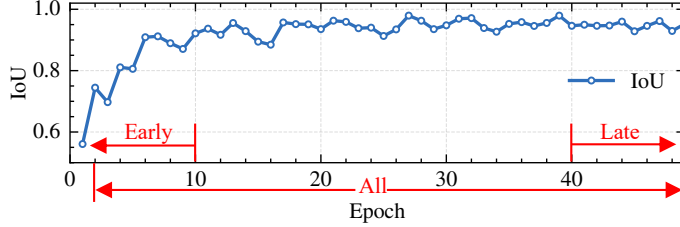
B. Suspiciousness Score and Ranking

After constructing box-level and image-level process features, ProAFL computes suspiciousness scores at both levels and integrates them into a unified inspection ranking. For box-level candidates, which correspond to existing GT boxes and are used to localize explicit annotation faults, ProAFL applies Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) [37] to aggregate the eight process features into one suspiciousness score. TOPSIS ranks candidates by their relative distances to the positive and negative ideal solutions, making it suitable for combining features with different scales and opposite directions of suspicion. A higher box-level score indicates a higher likelihood that the corresponding GT box contains an explicit annotation fault.

For image-level candidates, ProAFL also applies TOPSIS to score each prediction cluster used in missing-annotation localization. Since the inspection target is the image, ProAFL uses the highest cluster score in an image as its image-level suspiciousness score. Finally, because box-level and image-level scores are computed over candidate sets with different granularities and sizes, ProAFL introduces a **scaling factor** α to align image-level scores with box-level scores. Afterward, ProAFL merges all boxes and images, then sorts them by suspiciousness to produce a unified ranking for manual inspection.



(a) Training snapshots of one object at selected epochs (1, 5, 10, 40, 45, and 50), showing the matched prediction boxes in blue and the annotation box in green with their IoU values.



(b) IoU sequence of this object over the whole 50-epoch training process. We divide the sequence into three ranges: the first 10 epochs as Early, the last 10 epochs as Late, and all epochs as All.

$$\begin{aligned}
 IM &= \text{Mean}(IoU[:50]) = 0.919 \text{ (All)} \\
 EIM &= \text{Mean}(IoU[:10]) = 0.812 \text{ (Early)} \\
 LIM &= \text{Mean}(IoU[40:]) = 0.950 \text{ (Late)} \\
 DI &= \text{Min}\{e | IoU^e > 0.5 * LIM\} / 50 = 1/50
 \end{aligned}$$

(c) Process features computed from the IoU sequence.

Fig. 3: An example of box-level feature construction using IoU.

TABLE II: Definitions of the four image-level process features. $confidence_{cluster}$ denotes the confidence list of a prediction cluster, IoU_{matrix} denotes its pairwise IoU matrix, $class_{cluster}$ denotes predicted classes, N denotes the number of predicted boxes, $E_{cluster}$ denotes appearing epochs, and E_{last} denotes the late training epochs.

Process feature	Description	Equation
Cluster Confidence Mean (CCM)	Mean confidence in the cluster.	$CCM = \text{Mean}(confidence_{cluster})$
Cluster IoU Mean (CIM)	Spatial compactness of the cluster.	$CIM = \text{Mean}(IoU_{matrix})$
Cluster Class Consistency (CCC)	Class consistency of the cluster.	$CCC = \max(\text{count}(class_{cluster})) / N$
Cluster Epoch Span (CES)	Epoch persistence of the cluster.	$CES = E_{cluster} / E_{last}$

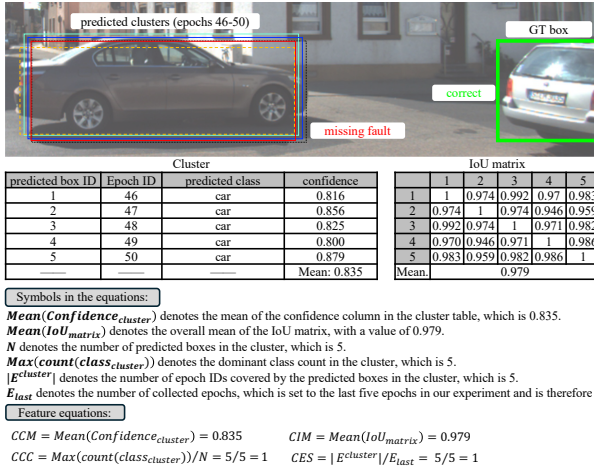


Fig. 4: An example of image-level feature construction

C. Correction and Retraining

Based on the unified rank, human reviewers inspect the top-ranked suspicious boxes or images within a given manual inspection budget and correct confirmed annotation faults.

For explicit faults, the correction process includes correcting misclassifications, adjusting inaccurate bounding boxes, and removing redundant boxes. For missing-annotation faults, human reviewers add annotations for real objects that were not originally labeled.

After annotation correction, ProAFL retrains the object detection model on the corrected training set. The repaired annotations provide cleaner supervision and reduce misleading training signals, enabling the retrained model to achieve better detection performance under the given inspection budget.

IV. EXPERIMENTAL SETUPS

This section describes our experimental setup. We evaluate whether ProAFL can effectively rank annotation faults and improve model performance following annotation correction and retraining. The two research questions are:

- RQ1: Can ProAFL effectively rank annotation faults?
- RQ2: Can ProAFL improve model performance through retraining?

The experiments were conducted on an Ubuntu 22.04.4 server equipped with two NVIDIA A800 GPUs with 80GB of memory. The deep learning frameworks included Python 3.10.19 and PyTorch 2.9.0 with CUDA 12.8.

TABLE III: Object detection datasets used in our experiments. #Class denotes the number of object categories, #Image denotes the number of images, and #Object denotes the number of annotated object instances.

Dataset	#Class	#Image		#Object	
		Train	Test	Train	Test
VOC	20	5,717	5,823	13,609	13,841
VisDrone	10	6,471	1,610	343,203	75,102
KITTI	8	3,740	3,741	20,423	20,147

A. Dataset and Models

We evaluate ProAFL on three object detection datasets: PASCAL VOC (VOC) [26], KITTI [27], and VisDrone [28]. As shown in Table III, these datasets differ substantially in the number of classes, images, and object instances. VOC is a widely used benchmark for generic object detection. KITTI focuses on autonomous driving scenarios, and we use its 2D object detection task [38]. VisDrone targets drone-based visual perception and contains a larger number of object instances, with dense objects, large-scale variations, and complex scenes. These datasets cover general, traffic, and aerial scenarios, enabling us to evaluate ProAFL under diverse data distributions and annotation complexities.

For models, to further examine the generality of ProAFL across different object detection model architectures, we use three representative models: YOLOv7 [29], Faster R-CNN (FRCNN) [30], and RT-DETR [31]. YOLOv7 is a one-stage detection model, FRCNN is a two-stage detection model, and RT-DETR represents real-time Transformer-based models.

B. Baselines

We compare ProAFL with six baseline methods that estimate the suspiciousness of annotations from different perspectives, including data decomposition, model loss, prediction uncertainty, and image-level label-quality rules. The baselines are introduced as follows.

(1) **Dactive** (ISSTA’24) [17] decomposes object detection annotations into foreground and background classification samples, and scores existing boxes and potential missing objects using robust classification loss.

(2) **Loss** (TNNLS’23) [32] ranks each annotated box by the combined classification and bounding-box regression loss of its matched prediction.

(3) **DeepGini** (ISSTA’20) [33] measures prediction uncertainty from the concentration of class probabilities and assigns the matched prediction’s score to the annotated box.

(4) **Entropy** (BMVC’18) [34] measures the uncertainty of the matched prediction’s class distribution, with higher entropy indicating higher suspiciousness.

(5) **Margin** [35] uses the gap between the two largest class probabilities of the matched prediction, where a smaller gap indicates higher suspiciousness.

(6) **ObjectLab** [16] computes image-level label quality scores by combining missing object, class, and localization

fault subtype scores from a trained detector and the original annotations.

Dactive and ObjectLab can identify images that may contain missing-annotation faults, whereas Loss, Entropy, Margin, and DeepGini operate on existing annotated boxes and therefore can only detect explicit annotation faults in principle. Since ObjectLab originally outputs image-level scores, we assign each image score to all GT boxes in that image to enable annotation-level comparison.

C. Data Fault Injection

Data fault injection is used to build a controlled evaluation setting in which faulty and correct annotations can be clearly distinguished. Since manually reviewing all samples in large-scale object detection datasets is costly, we follow the experimental setting of Dactive [17] and manually inject annotation faults into existing object detection datasets. This produces noisy training sets with known fault locations for evaluation.

Annotation faults in object detection involve not only class labels but also bounding-box locations and the existence of an annotation box. We inject two types of faults. The first type is explicit faults, including class faults, localization faults, and redundant faults. The second type is implicit faults, i.e., missing faults. Specifically, a class fault means that the class label of an annotated box is incorrect; a localization fault means that the annotated box is shifted from the true object location; a redundant fault means that an extra false box is added to the image; and a missing fault means that a real object is not annotated.

In implementation, class faults are generated by modifying only the class label. Localization faults are generated by perturbing the original bounding box until its IoU with the original box falls between 0.1 and 0.5. Redundant faults are generated by adding false object boxes at random positions in the image. Missing faults are generated by deleting the corresponding object annotations. For each fault type, we randomly sample approximately 10% of object instances and rewrite their annotations according to the target fault type, thereby obtaining explicit ground truth for ranking evaluation.

D. Experimental settings

We describe the key experimental settings for the three steps of ProAFL, as detailed in Section III. **In Step 1**, YOLOv7 and RT-DETR are trained for 50 and 120 epochs, respectively. Predictions are matched with GT boxes using an IoU threshold of 0.5. For box-level features, the early and late stages are defined as the first 20% and the last 20% of training epochs, respectively. For image-level features, ProAFL collects high-confidence unmatched predictions from the last five epochs with confidence greater than 0.6. Prediction clustering uses an IoU threshold of 0.6, i.e., two predictions are merged into the same cluster when their pairwise IoU exceeds 0.6. **In Step 2**, for score alignment, the default scaling factor is set to $\alpha = 1.5$. **In Step 3**, the inspection budget is set to the top 50% of

the generated ranking, and post-correction retraining uses 25 additional epochs.

E. Evaluation Indicators

We evaluate ProAFL using six indicators from two aspects: ranking effectiveness and retraining performance. The first five indicators measure ranking effectiveness, while the last one measures model performance after retraining.

(1) **APFD** [39] measures global ranking quality, i.e., whether real faults appear early in the inspection list:

$$\text{APFD} = 1 - \frac{\sum_{i=1}^n (\text{rank}_i - 0.5)}{n \times m} + \frac{1}{2m} \quad (1)$$

where n is the number of real faults, m is the number of ranked inspection units, and rank_i is the rank of the i -th fault. A higher APFD is better.

(2) **FNR** [40] evaluates fault coverage under the top-50% inspection budget. It is the ratio of real faults missed by the inspected candidates; a lower FNR is better.

(3) **FPR** [40] evaluates inspection precision under the top-50% inspection budget. It is the ratio of inspected candidates that are actually correct; a lower FPR is better.

(4) **Top-1** [41] evaluates within-image localization by checking whether the first inspected candidate in an erroneous image is faulty. A higher Top-1 is better.

(5) **EXAM** [42] records the normalized rank of the first faulty candidate within each erroneous image. A lower EXAM indicates less inspection effort.

(6) **mAP@0.5** [26] evaluates post-correction detection performance. After correcting the inspected annotations, we retrain the detector and report mAP at IoU 0.5; a higher value indicates better performance.

V. EXPERIMENT RESULTS

In this section, we report the experimental results and answer the two research questions.

RQ1: Can ProAFL effectively rank annotation faults?

Motivation and method. This RQ aims to evaluate whether ProAFL can effectively localize faulty annotations in training sets and generate reliable rankings of suspiciousness. We compare ProAFL with six baseline methods, including Dactive, Entropy, Loss, DeepGini, Margin, and ObjectLab, on three datasets and three object detection models, yielding nine dataset-model scenarios. In each scenario, ProAFL and the baselines generate an annotation suspiciousness ranking. We use APFD to evaluate overall ranking quality, and FNR and FPR to measure ranking effectiveness under a fixed inspection budget, where the top 50% of the ranking is used as the inspection budget, and Top-1 and EXAM to evaluate within-image localization efficiency.

Results. Table IV shows that ProAFL achieves the best average performance on all five indicators. The gray cells mark the best result among all methods under each Indicator-Dataset-Model setting. Specifically, compared with the best baseline averages, ProAFL improves APFD from 0.693 to 0.768, reduces FNR from 0.217 to 0.070, reduces FPR from

TABLE IV: Ranking performance of ProAFL and the baselines. $\uparrow/\downarrow$ means higher/lower is better. Dactive is model-agnostic. Gray cells mark the best result in each dataset-model-indicator scenario. Average is computed over nine scenarios, and #Best/9 reports, for each method, the number of scenarios where it achieves the best result for each indicator.

Indicator	Dataset	Model	ProAFL	Dactive	Entropy	Loss	DeepGini	Margin	ObjectLab
APFD $\uparrow$	VOC	YOLOv7	0.779		0.679	0.699	0.681	0.683	0.462
		FRCNN	0.751	0.678	0.669	0.600	0.674	0.673	0.465
		RT-DETR	0.735		0.629	0.630	0.629	0.631	0.510
	KITTI	YOLOv7	0.790		0.631	0.703	0.632	0.633	0.489
		FRCNN	0.784	0.678	0.672	0.614	0.672	0.672	0.495
		RT-DETR	0.810		0.609	0.645	0.611	0.617	0.575
	VisDrone	YOLOv7	0.746		0.626	0.690	0.623	0.624	0.446
		FRCNN	0.744	0.722	0.625	0.669	0.626	0.630	0.541
		RT-DETR	0.777		0.633	0.734	0.634	0.641	0.476
	Average #Best/9		0.768 9	0.693 0	0.641 0	0.665 0	0.642 0	0.645 0	0.495 0
FNR $\downarrow$	VOC	YOLOv7	0.084		0.269	0.257	0.272	0.268	0.481
		FRCNN	0.101	0.300	0.283	0.347	0.279	0.277	0.611
		RT-DETR	0.165		0.318	0.325	0.318	0.309	0.419
	KITTI	YOLOv7	0.014		0.335	0.204	0.333	0.332	0.466
		FRCNN	0.032	0.179	0.241	0.189	0.239	0.238	0.505
		RT-DETR	0.016		0.341	0.310	0.336	0.321	0.334
	VisDrone	YOLOv7	0.068		0.275	0.123	0.282	0.279	0.656
		FRCNN	0.093	0.172	0.287	0.189	0.284	0.273	0.454
		RT-DETR	0.059		0.316	0.104	0.315	0.299	0.547
	Average #Best/9		0.070 9	0.217 0	0.296 0	0.243 0	0.295 0	0.288 0	0.497 0
FPR $\downarrow$	VOC	YOLOv7	0.344		0.413	0.409	0.414	0.413	0.502
		FRCNN	0.350	0.434	0.419	0.443	0.417	0.416	0.551
		RT-DETR	0.374		0.431	0.434	0.432	0.428	0.479
	KITTI	YOLOv7	0.277		0.424	0.364	0.423	0.423	0.486
		FRCNN	0.285	0.354	0.381	0.422	0.380	0.380	0.504
		RT-DETR	0.278		0.427	0.412	0.425	0.418	0.425
	VisDrone	YOLOv7	0.304		0.398	0.330	0.401	0.400	0.571
		FRCNN	0.316	0.351	0.404	0.359	0.402	0.397	0.479
		RT-DETR	0.300		0.417	0.321	0.416	0.409	0.521
	Average #Best/9		0.314 9	0.380 0	0.413 0	0.388 0	0.412 0	0.409 0	0.502 0
Top-1 $\uparrow$	VOC	YOLOv7	0.793		0.826	0.860	0.830	0.831	0.385
		FRCNN	0.712	0.538	0.829	0.667	0.832	0.834	0.301
		RT-DETR	0.881		0.775	0.777	0.775	0.779	0.281
	KITTI	YOLOv7	0.658		0.779	0.872	0.783	0.782	0.476
		FRCNN	0.757	0.613	0.832	0.750	0.828	0.829	0.469
		RT-DETR	0.924		0.711	0.804	0.717	0.712	0.473
	VisDrone	YOLOv7	0.902		0.627	0.779	0.627	0.627	0.903
		FRCNN	0.944	0.822	0.626	0.858	0.621	0.624	0.928
		RT-DETR	0.868		0.745	0.882	0.745	0.746	0.926
	Average #Best/9		0.827 4	0.658 0	0.750 1	0.805 2	0.751 0	0.752 1	0.571 1
EXAM $\downarrow$	VOC	YOLOv7	0.457		0.511	0.494	0.510	0.509	0.611
		FRCNN	0.477	0.473	0.508	0.556	0.506	0.505	0.644
		RT-DETR	0.463		0.530	0.527	0.530	0.530	0.580
	KITTI	YOLOv7	0.318		0.474	0.424	0.472	0.472	0.531
		FRCNN	0.313	0.381	0.438	0.478	0.439	0.438	0.532
		RT-DETR	0.294		0.484	0.461	0.482	0.476	0.452
	VisDrone	YOLOv7	0.174		0.407	0.366	0.407	0.407	0.308
		FRCNN	0.134	0.154	0.372	0.348	0.369	0.369	0.222
		RT-DETR	0.215		0.380	0.343	0.380	0.380	0.254
	Average #Best/9		0.316 8	0.336 1	0.456 0	0.444 0	0.455 0	0.454 0	0.459 0

0.380 to 0.314, improves Top-1 from 0.805 to 0.827, and reduces EXAM from 0.336 to 0.316. In terms of the best scene count (see the ‘#Best/9’ row), ProAFL achieves the best APFD, FNR, and FPR in all nine scenarios, the best Top-1 in four scenarios, and the best EXAM in eight scenarios. These results indicate that ProAFL achieves better ranking performance and better fault localization effectiveness under the same inspection budget.

Answer to RQ1: ProAFL achieves better annotation fault ranking performance than the baselines.

TABLE V: Retraining performance under the top 50% inspection budget. YOLOv7 is retrained on the corrected training set and evaluated on the clean test set using mAP@0.5. Higher values are better. Gray cells mark the best method.

Dataset	Model	ProAFL	Dactive	Entropy	Loss	DeepGini	Margin	ObjectLab
VOC	YOLOv7	0.798	0.790	0.780	0.773	0.789	0.783	0.761
KITTI		0.898	0.891	0.876	0.874	0.869	0.872	0.840
VisDrone		0.392	0.389	0.383	0.390	0.382	0.388	0.381
Average	–	0.696	0.690	0.680	0.679	0.680	0.681	0.661

TABLE VI: Average ranking performance of ProAFL under different scaling factors on VOC2012, KITTI, and VisDrone. $\uparrow/\downarrow$ means higher/lower is better. Std. denotes the standard deviation across factors. Gray cells mark the best result.

Metric	0.5	1.0	1.5	2.0	Std.
APFD $\uparrow$	0.764	0.781	0.772	0.769	0.0061
FPR $\downarrow$	0.324	0.310	0.308	0.308	0.0067
FNR $\downarrow$	0.091	0.058	0.055	0.143	0.0354
Top-1 $\uparrow$	0.866	0.875	0.784	0.768	0.0477
EXAM $\downarrow$	0.356	0.325	0.316	0.319	0.0159

RQ2: Can ProAFL improve model performance through re-training?

Motivation and method. RQ1 (see Section V) evaluates whether ProAFL can generate effective annotation fault rankings. The purpose of such rankings is to improve manual inspection efficiency: faulty annotations should be corrected at a limited inspection cost, and the corrected training set should further improve model performance upon retraining. To evaluate this effect, we compare ProAFL with the baselines on YOLOv7 across the three datasets. For each method, we inspect and correct the top 50% of candidates by rank, using the same annotation-box-level correction cost for all fault types. We then retrain YOLOv7 on the corrected training set and evaluate the retrained model on the corresponding clean test set using mAP@0.5.

Results. Table V shows that ProAFL achieves the highest retrained mAP@0.5 on all three datasets. Specifically, ProAFL obtains 0.798 on VOC and 0.898 on KITTI, outperforming Dactive by 0.008 and 0.007, respectively; on VisDrone, it reaches 0.392, slightly higher than the best baseline Loss at 0.390. On average, ProAFL achieves 0.696 mAP@0.5, outperforming the best baseline average of 0.690. These results indicate that ProAFL can more effectively translate a limited annotation-correction budget into improved performance of the retrained model.

Answer to RQ2: ProAFL achieves the best performance of the retrained model under the same inspection budget.

VI. DISCUSSION

A. The effect of scaling factor on sorting and merging

Motivation and method. As stated in Section III-B, ProAFL needs to align box-level and image-level suspiciousness scores

before merging them into a unified ranking. We introduce a scaling factor α for this alignment, which controls the relative priority of image-level and box-level candidates in the final ranking. A small α may lower the priority of image-level candidates, whereas a large α may promote them and suppress box-level candidates. Therefore, we analyze how α affects the final ranking performance.

We keep all other settings fixed and vary only α over four values: 0.5, 1.0, 1.5, and 2.0. Using YOLOv7 on VOC2012, KITTI, and VisDrone, we report the mean values of the five indicators defined in RQ1 (Section V) for each α , as well as the standard deviation (Std.) across the four α values.

Results. Table VI shows that ProAFL is not sensitive to the scaling factor α . Since these indicators are normalized to the range of 0 to 1, Std. values below 0.05 correspond to fluctuations within five percentage points. This suggests that changing α from 0.5 to 2.0 has only a limited impact on ranking performance. Among the four values, $\alpha = 1.5$ achieves the best or tied-best performance on three indicators: FPR, FNR, and EXAM. Therefore, we set $\alpha = 1.5$ as the default value in ProAFL.

B. ProAFL’s ability to locate different fault types

Motivation and method. As stated in Section IV-C, we inject four types of annotation faults: class faults, localization faults, redundant faults, and missing faults. Therefore, we further analyze whether ProAFL can effectively localize each fault type, rather than only improving ranking indicators.

We conduct this analysis with YOLOv7 on VOC2012, KITTI, and VisDrone. For each dataset, we scan the suspiciousness ranking generated by ProAFL from top to bottom and compute the cumulative detection ratio of each fault type under different inspection ratios.

Results. The results are shown in Fig. 5. As the inspection ratio increases, the detection ratios for different fault types steadily rise across the three datasets, and the cumulative detection ratio consistently exceeds the theoretical random-inspection baseline. These results indicate that ProAFL can effectively prioritize different types of annotation faults in the ranking.

C. Importance of Process Features

Motivation and method. As described in Section III-A, ProAFL constructs eight box-level process features and four image-level process features to compute suspiciousness scores. However, different features may contribute differently to distinguishing correct annotations from faulty annotations. Therefore, this section further analyzes the importance of each process feature.

We conduct this analysis using YOLOv7 on the VOC, KITTI, and VisDrone datasets. For each process feature, we compute the absolute Spearman correlation coefficient [43] between the feature value and the binary fault label as its importance score. Specifically, for box-level features, we measure their ability to distinguish explicit faulty annotations from correct ones; for image-level features, we measure their ability

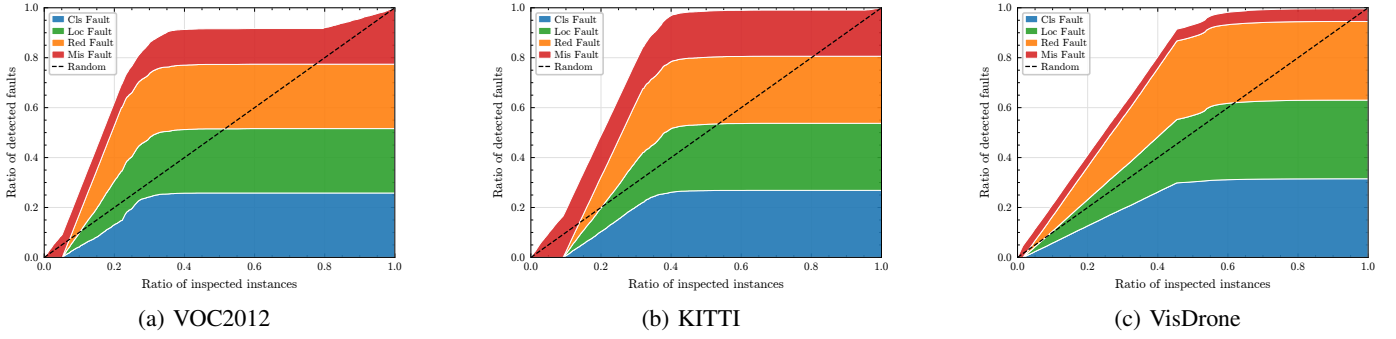


Fig. 5: Cumulative detection results of different annotation-fault types by ProAFL. The three subplots show the results on VOC2012, KITTI, and VisDrone under YOLOv7. The x-axis denotes the inspected candidate ratio, and the y-axis denotes the cumulative fault detection ratio. Different colored areas indicate the contributions of class faults, localization faults, redundant boxes, and missing annotations.

TABLE VII: Spearman correlation between process features and fault labels. The values in the table are the absolute Spearman correlation coefficients. Gray cells indicate the most important feature within the box level or image level for each dataset.

Dataset	Box-level features								Image-level features			
	ECM	EIM	LCM	LIM	CM	IM	DC	DI	CCM	CIM	CCC	CES
VOC2012	0.731	0.736	0.746	0.732	0.775	0.779	0.755	0.774	0.379	0.351	0.368	0.378
KITTI	0.720	0.725	0.749	0.749	0.768	0.772	0.724	0.742	0.440	0.383	0.462	0.460
VisDrone	0.513	0.516	0.580	0.582	0.597	0.600	0.570	0.572	0.161	0.099	0.225	0.232

to distinguish images with missing annotations from those without.

Results. The results are shown in Table VII. For the eight box-level features defined in Section III-A, IM achieves the highest importance score on all three datasets, indicating that the mean IoU over the full training process is the most stable feature for distinguishing explicit annotation faults. For the four image-level features defined in Section III-A, the most important feature varies across datasets: CCM on VOC2012, CCC on KITTI, and CES on VisDrone. This suggests that the importance of image-level process features for identifying missing annotations depends more on dataset-specific scene characteristics and object distributions.

D. Overlap between our method and baseline methods

Motivation and method. To further understand the relationship between ProAFL and existing baselines in annotation-fault localization, we analyze the overlap between the true faults detected by ProAFL and those detected by each baseline method, as well as by the union of all baselines.

Specifically, using YOLOv7 on VOC2012, KITTI, and VisDrone, we inspect the top 50% units in each method’s ranking and compare the true faults detected by ProAFL with those detected by each baseline method and by the union of all baselines. For each comparison, we report Ours only, Overlap, and Baseline only, denoting the true faults detected only by ProAFL, by both methods, and only by the corresponding baseline method or baseline union, respectively.

TABLE VIII: Overlap between the true faults detected by ProAFL and the baselines when inspecting the top 50% units in each ranking. Percentages are relative to the total number of true faults. All Baselines denotes the union of all baseline detections.

Dataset	Comparison	Ours only	Overlap	Baseline only
VOC2012	Dactive	1,711 (32.46%)	3,117 (59.13%)	305 (5.79%)
	Entropy	978 (18.55%)	3,850 (73.04%)	5 (0.09%)
	Loss	919 (17.44%)	3,909 (74.16%)	5 (0.09%)
	DeepGini	994 (18.86%)	3,834 (72.74%)	5 (0.09%)
	Margin	976 (18.52%)	3,852 (73.08%)	5 (0.09%)
	ObjectLab	2,608 (49.48%)	2,220 (42.12%)	246 (4.67%)
	All Baselines	147 (2.79%)	4,681 (88.81%)	315 (5.98%)
KITTI	Dactive	1,386 (18.23%)	6,111 (80.39%)	81 (1.07%)
	Entropy	2,454 (32.28%)	5,043 (66.34%)	15 (0.20%)
	Loss	1,474 (19.39%)	6,023 (79.23%)	32 (0.42%)
	DeepGini	2,439 (32.08%)	5,058 (66.54%)	16 (0.21%)
	Margin	2,433 (32.00%)	5,064 (66.61%)	16 (0.21%)
	ObjectLab	3,583 (47.13%)	3,914 (51.49%)	96 (1.26%)
	All Baselines	47 (0.62%)	7,450 (98.00%)	103 (1.35%)
VisDrone	Dactive	14,720 (13.51%)	86,832 (79.71%)	3,410 (3.13%)
	Entropy	24,513 (22.50%)	77,039 (70.72%)	1,946 (1.79%)
	Loss	10,062 (9.24%)	91,490 (83.99%)	4,024 (3.69%)
	DeepGini	25,090 (23.03%)	76,462 (70.19%)	1,808 (1.66%)
	Margin	24,827 (22.79%)	76,725 (70.43%)	1,821 (1.67%)
	ObjectLab	70,246 (64.48%)	31,306 (28.74%)	6,167 (5.66%)
	All Baselines	1 (0.00%)	101,551 (93.22%)	7,366 (6.76%)

Results. Table VIII shows the overlap between true faults detected by ProAFL and those detected by the baselines. Each cell is reported as $x(y\%)$, where x denotes the number of true positives and y denotes its percentage among true faults. Compared with individual baselines, ProAFL detects additional true faults, especially over ObjectLab, where the Ours only ratios reach 49.48%, 47.13%, and 64.48% on VOC2012, KITTI, and VisDrone, respectively. Compared with the union of all baselines, ProAFL shows overlap ratios of 88.81%, 98.00%, and 93.22%, while the Baseline only ratios are 5.98%, 1.35%, and 6.76%. These results indicate that ProAFL captures most of the fault signals covered by the baselines and provides complementary detections compared with individual methods.

TABLE IX: Box-level localization ability of ProAFL for missing annotations. The table reports Recall under different IoU thresholds. A real missing box is counted as localized if it is hit by at least one unmatched prediction cluster with $\text{IoU} > t \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$. Higher values indicate stronger spatial coverage of missing objects.

Dataset	$\text{IoU} > 0.5$	$\text{IoU} > 0.6$	$\text{IoU} > 0.7$	$\text{IoU} > 0.8$	$\text{IoU} > 0.9$
VOC2012	0.496	0.490	0.474	0.432	0.333
KITTI	0.769	0.761	0.746	0.697	0.461
VisDrone	0.276	0.273	0.264	0.231	0.126

E. Missing fault box-level location

Motivation and method. Previous experiments show that ProAFL can identify images that may contain missing annotations. However, practical review can be further improved if the method also provides approximate locations of missing objects. Since the unmatched prediction clusters collected in Section III-A preserve spatial information, we further evaluate whether they can localize real missing objects.

We conduct this analysis using YOLOv7 on the VOC, KITTI, and VisDrone datasets. Each prediction cluster is treated as a candidate missing-object region and matched with the real missing boxes in the same image. Given an IoU threshold t , a cluster hits a missing object if at least one prediction box in the cluster has an IoU greater than t with a real missing box. A missing box is considered localized if it is hit by at least one cluster. We report Recall under $t \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$.

Results. Table IX shows that unmatched prediction clusters can provide useful spatial cues for missing annotations. At IoU 0.5, ProAFL achieves Recall values of 0.496, 0.769, and 0.276 on VOC2012, KITTI, and VisDrone, indicating that the generated clusters can cover many missing objects. As the IoU threshold increases, Recall decreases on all datasets, especially at IoU 0.9. This suggests that these clusters are better suited as region-level hints for manual review than as direct substitutes for precise bounding boxes.

VII. THREAD TO VALIDITY

This section discusses the threats to the validity of our approach. **(1) Datasets.** Object detection models may show different training processes on different datasets. We mitigate this threat by using three datasets with different scenarios: VOC for general object detection, KITTI for autonomous driving scenes, and VisDrone for dense aerial objects. Nevertheless, additional datasets and naturally noisy data are needed to validate generalizability further. **(2) Models.** Similarly, different detectors may produce different training-process signals. We reduce this threat by evaluating ProAFL with three representative architectures: YOLOv7, Faster R-CNN, and RT-DETR. Future work should examine more models with different architectures. **(3) Data fault simulation.** We use simulated annotation faults because large-scale object detection datasets with verified real annotation faults are difficult to obtain. Following prior work, we inject four common

fault types: class faults, localization faults, redundant boxes, and missing annotations. Although this enables controlled evaluation, simulated faults may not fully reflect real fault distributions; future work should further validate ProAFL on manually verified annotation errors.

VIII. RELATEDWORK

Annotation Debugging for Object Detection. Some studies on noisy object detection mitigate the impact of noisy datasets by adjusting or refining annotations during training, such as adaptive learning under noisy annotations [44], self-correction for noisy box locations [45], and iterative annotation refinement [46]. Other studies directly prioritize suspicious annotations for inspection: Dataactive decomposes object detection data into foreground/background classification samples [17], ObjectLab estimates image-level label quality from a trained detector [16], Loss Inspection uses detector losses to identify annotation faults [47], and CLIBAD uses anomaly descriptions generated by language models to localize annotation errors [48]. *Different from these studies, ProAFL localizes annotation faults from the process metrics perspective by modeling cross-epoch learning behaviors of annotations and constructing process features from training process information.*

Data Quality in Deep Learning. Sambasivan et al. [6] showed that data problems can cascade through the AI lifecycle, while studies on pipeline debugging further revealed that preprocessing, distribution shifts, and data issues related to features can affect model behavior [49], [50]. Other work localizes training data factors behind fairness problems [14], perception errors [51], and label correctness issues in software analytics [8].

Noisy Label Learning and Classification Fault Localization. Many studies mitigate noisy training through training strategies or model optimization [52]–[58]. Other studies directly localize noisy or incorrect labels, mainly in classification tasks, by using predicted probabilities, task specific label quality estimation, crowdsourcing based noise correction, or data fault localization techniques [15], [59]–[62].

IX. CONCLUSION

This paper proposes ProAFL, a process metrics-based method for localizing annotation faults in object detection datasets. Unlike prior methods that rely on final model outputs or data decomposition, ProAFL uses process metrics to construct process features for annotating faults. It ranks suspicious annotations for manual inspection under limited budgets and supports model retraining after annotation correction. Experiments on three datasets and three models show that ProAFL improves fault ranking and retraining performance. These results demonstrate the effectiveness of process-metric information for debugging data in object detection systems.

A. Replication Package

We present the dataset, source code files, and details online at <https://github.com/ProAFL/ProAFL>.

REFERENCES

- [1] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, 2023.
- [2] M. Liang, J. Su, S. Schuster, S. Garg, S. Zhao, Y. Wu, and M. Chandraker, "AIDE: an automatic data engine for object detection in autonomous driving," in *CVPR*. IEEE, 2024, pp. 14 695–14 706.
- [3] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in *CVPR*. IEEE, 2023, pp. 17 853–17 862.
- [4] S. Wang, S. Li, Y. Zhang, S. Yu, S. Yuan, R. She, Q. Guo, J. Zheng, O. K. Howe, L. Chandra, S. Srijevan, A. Sivasdas, T. Aggarwal, H. Liu, H. Zhang, C. Chen, J. Jiang, L. Xie, and W. P. Tay, "Uavscenes: A multi-modal dataset for uavs," in *ICCV*. IEEE, 2025, pp. 28 946–28 958.
- [5] S. Xu, Y. Yao, F. Xu, T. Gu, J. Xu, and X. Ma, "Data quality matters: A case study of obsolete comment detection," in *ICSE*. IEEE, 2023, pp. 781–793.
- [6] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. K. Paritosh, and L. Aroyo, "'everyone wants to do the model work, not the data work': Data cascades in high-stakes AI," in *CHI*. ACM, 2021, pp. 39:1–39:15.
- [7] T. Lin, M. Maire, S. J. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: common objects in context," in *ECCV (5)*, ser. Lecture Notes in Computer Science, vol. 8693. Springer, 2014, pp. 740–755.
- [8] X. Wu, W. Zheng, X. Xia, and D. Lo, "Data quality matters: A case study on data label correctness for security bug report prediction," *IEEE Trans. Software Eng.*, vol. 48, no. 7, pp. 2541–2556, 2022.
- [9] J. Chakraborty, S. Majumder, Z. Yu, and T. Menzies, "Fairway: a way to build fair ML software," in *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, P. Devanbu, M. B. Cohen, and T. Zimmermann, Eds. ACM, 2020, pp. 654–665. [Online]. Available: <https://doi.org/10.1145/3368089.3409697>
- [10] X. Zhang, X. Xie, L. Ma, X. Du, Q. Hu, Y. Liu, J. Zhao, and M. Sun, "Towards characterizing adversarial defects of deep learning software from the lens of uncertainty," in *ICSE*. ACM, 2020, pp. 739–751.
- [11] Y. Zhang, Y. Wang, H. Zhang, B. Zhu, S. Chen, and D. Zhang, "Onelabeler: A flexible system for building data labeling tools," in *CHI*. ACM, 2022, pp. 93:1–93:22.
- [12] M. Burges, P. A. Dias, C. Woody, S. Walters, and D. D. Lunga, "Active learning meets foundation models: Fast remote sensing data annotation for object detection," in *ICCV*. IEEE, 2025, pp. 6058–6068.
- [13] A. Ghorbani and J. Y. Zou, "Data shapley: Equitable valuation of data for machine learning," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 2019, pp. 2242–2251. [Online]. Available: <http://proceedings.mlr.press/v97/ghorbani19c.html>
- [14] Y. Li, L. Meng, L. Chen, L. Yu, D. Wu, Y. Zhou, and B. Xu, "Training data debugging for the fairness of machine learning software," in *ICSE*. ACM, 2022, pp. 2215–2227.
- [15] Y. Yin, Y. Feng, S. Weng, Z. Liu, Y. Yao, Y. Zhang, Z. Zhao, and Z. Chen, "Dynamic data fault localization for deep neural networks," in *ESEC/SIGSOFT FSE*. ACM, 2023, pp. 1345–1357.
- [16] U. Tkachenko, A. Thyagarajan, and J. Mueller, "Objectlab: Automated diagnosis of mislabeled images in object detection data," *CoRR*, vol. abs/2309.00832, 2023.
- [17] Y. Yin, Y. Feng, S. Weng, Y. Yao, J. Liu, and Z. Zhao, "Dataactive: Data fault localization for object detection systems," in *ISSA*. ACM, 2024, pp. 895–907.
- [18] A. E. Hassan, "Predicting faults using the complexity of code changes," in *ICSE*. IEEE, 2009, pp. 78–88.
- [19] N. Nagappan and T. Ball, "Use of relative code churn measures to predict system defect density," in *ICSE*. ACM, 2005, pp. 284–292.
- [20] R. Moser, W. Pedrycz, and G. Succi, "A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction," in *ICSE*. ACM, 2008, pp. 181–190.
- [21] F. Rahman and P. T. Devanbu, "How, and why, process metrics are better," in *ICSE*. IEEE Computer Society, 2013, pp. 432–441.
- [22] N. M. Synovic, M. Hyatt, R. Sethi, S. Thota, Shilpika, A. J. Miller, W. Jiang, E. S. Amobi, A. Pinderski, K. Läufer, N. J. Hayward, N. Klingensmith, J. C. Davis, and G. K. Thiruvathukal, "Snapshot metrics are not enough: Analyzing software repositories with longitudinal metrics," in *ASE*. ACM, 2022, pp. 167:1–167:4.
- [23] S. Tabassum, L. L. Minku, D. Feng, G. G. Cabral, and L. Song, "An investigation of cross-project learning in online just-in-time software defect prediction," in *ICSE*. ACM, 2020, pp. 554–565.
- [24] Z. Li, X. Ma, C. Xu, J. Xu, C. Cao, and J. Lu, "Operational calibration: debugging confidence errors for dnns in the field," in *ESEC/SIGSOFT FSE*. ACM, 2020, pp. 901–913.
- [25] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. D. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *CVPR*. Computer Vision Foundation / IEEE, 2019, pp. 658–666.
- [26] M. Everingham, L. V. Gool, C. K. I. Williams, J. M. Winn, and A. Zisserman, "The pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.
- [27] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the KITTI vision benchmark suite," in *CVPR*. IEEE Computer Society, 2012, pp. 3354–3361.
- [28] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and tracking meet drones challenge," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7380–7399, 2022.
- [29] C. Wang, A. Bochkovskiy, and H. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *CVPR*. IEEE, 2023, pp. 7464–7475.
- [30] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, Eds., 2015, pp. 91–99. [Online]. Available: <https://proceedings.neurips.cc/paper/2015/hash/14bfa6bb14875e45bba028a21ed38046-Abstract.html>
- [31] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "Detrs beat yolos on real-time object detection," in *CVPR*. IEEE, 2024, pp. 16 965–16 974.
- [32] H. Song, M. Kim, D. Park, Y. Shin, and J. Lee, "Learning from noisy labels with deep neural networks: A survey," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 34, no. 11, pp. 8135–8153, 2023.
- [33] Y. Feng, Q. Shi, X. Gao, J. Wan, C. Fang, and Z. Chen, "Deepgini: prioritizing massive tests to enhance the robustness of deep neural networks," in *ISSA '20: 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, USA, July 18-22, 2020*, S. Khurshid and C. S. Pasareanu, Eds. ACM, 2020, pp. 177–188. [Online]. Available: <https://doi.org/10.1145/3395363.3397357>
- [34] S. Roy, A. Unmesh, and V. P. Nambodiri, "Deep active learning for object detection," in *BMVC*. BMVA Press, 2018, p. 91.
- [35] C. Brust, C. Käding, and J. Denzler, "Active learning for deep object detection," *CoRR*, vol. abs/1809.09875, 2018. [Online]. Available: <http://arxiv.org/abs/1809.09875>
- [36] H. N. Gabow and R. E. Tarjan, "A linear-time algorithm for a special case of disjoint set union," in *STOC*. ACM, 1983, pp. 246–251.
- [37] M. Behzadian, S. K. Otahsara, M. Yazdani, and J. Ignatius, "A state-of-the-art survey of topsiis applications," *Expert Systems with applications*, vol. 39, no. 17, pp. 13 051–13 069, 2012.
- [38] The kitti vision benchmark suite. [Online]. Available: https://www.cvlibs.net/datasets/kitti/eval_object.php?obj_benchmark
- [39] S. Yoo, A. M. Harman, "Regression testing minimization, selection and prioritization: a survey," *Softw. Test. Verification Reliab.*, vol. 22, no. 2, pp. 67–120, 2012.
- [40] Q. Song, M. J. Shepperd, M. Cartwright, and C. Mair, "Software defect association mining and defect correction effort prediction," *IEEE Trans. Software Eng.*, vol. 32, no. 2, pp. 69–82, 2006.
- [41] P. S. Kochhar, X. Xia, D. Lo, and S. Li, "Practitioners' expectations on automated fault localization," in *ISSA*. ACM, 2016, pp. 165–176.
- [42] S. Pearson, J. Campos, R. Just, G. Fraser, R. Abreu, M. D. Ernst, D. Pang, and B. Keller, "Evaluating and improving fault localization," in *ICSE*. IEEE / ACM, 2017, pp. 609–620.
- [43] M. Beller, G. Gousios, A. Panichella, and A. Zaidman, "When, how, and why developers (do not) test in their ides," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 179–190.
- [44] X. Liu, W. Li, Q. Yang, B. Li, and Y. Yuan, "Towards robust adaptive object detection under noisy annotations," in *CVPR*. IEEE, 2022, pp. 14 187–14 196.

- [45] S. Wang, J. Gao, B. Li, and W. Hu, "Narrowing the gap: Improved detector training with noisy location annotations," *IEEE Trans. Image Process.*, vol. 31, pp. 6369–6380, 2022.
- [46] J. Mao, Q. Yu, Y. Yamakata, and K. Aizawa, "Noisy annotation refinement for object detection," in *BMVC*. BMVA Press, 2021.
- [47] M. Schubert, T. Riedlinger, K. Kahl, D. Kröll, S. Schoenen, S. Segvic, and M. Rottmann, "Identifying label errors in object detection datasets by loss inspection," in *WACV*. IEEE, 2024, pp. 4570–4579.
- [48] H. Hu, "Clibad: Zero-shot localization for annotation error in object detection datasets," in *2025 8th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE)*. IEEE, 2025, pp. 333–338.
- [49] S. Grafberger, P. Groth, J. Stoyanovich, and S. Schelter, "Data distribution debugging in machine learning pipelines," *VLDB J.*, vol. 31, no. 5, pp. 1103–1126, 2022.
- [50] B. Karlas, D. Dao, M. Interlandi, B. Li, S. Schelter, W. Wu, and C. Zhang, "Data debugging with shapley importance over end-to-end machine learning pipelines," *CoRR*, vol. abs/2204.11131, 2022.
- [51] D. Kang, N. Aréchiga, S. Pillai, P. D. Bailis, and M. Zaharia, "Finding label and model errors in perception data with learned observation assertions," in *SIGMOD Conference*. ACM, 2022, pp. 496–505.
- [52] D. Arpit, S. Jastrzebski, N. Ballas, D. Krueger, E. Bengio, M. S. Kanwal, T. Maharaj, A. Fischer, A. C. Courville, Y. Bengio, and S. Lacoste-Julien, "A closer look at memorization in deep networks," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 233–242.
- [53] P. Chen, B. Liao, G. Chen, and S. Zhang, "Understanding and utilizing deep neural networks trained with noisy labels," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 1062–1070.
- [54] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. W. Tsang, and M. Sugiyama, "Co-teaching: Robust training of deep neural networks with extremely noisy labels," in *NeurIPS*, 2018, pp. 8536–8546.
- [55] L. Jiang, Z. Zhou, T. Leung, L. Li, and L. Fei-Fei, "Mentornet: Learning data-driven curriculum for very deep neural networks on corrupted labels," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 2309–2318.
- [56] A. Ghosh, H. Kumar, and P. S. Sastry, "Robust loss functions under label noise for deep neural networks," in *AAAI*. AAAI Press, 2017, pp. 1919–1925.
- [57] Z. Zhang and M. R. Sabuncu, "Generalized cross entropy loss for training deep neural networks with noisy labels," in *NeurIPS*, 2018, pp. 8792–8802.
- [58] J. Goldberger and E. Ben-Reuven, "Training deep neural-networks using a noise adaptation layer," in *ICLR (Poster)*. OpenReview.net, 2017.
- [59] C. G. Northcutt, L. Jiang, and I. L. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *J. Artif. Intell. Res.*, vol. 70, pp. 1373–1411, 2021.
- [60] A. Thyagarajan, E. Snorrason, C. G. Northcutt, and J. Mueller, "Identifying incorrect annotations in multi-label classification data," *CoRR*, vol. abs/2211.13895, 2022.
- [61] W. Wang and J. Mueller, "Detecting label errors in token classification data," *CoRR*, vol. abs/2210.03920, 2022.
- [62] J. Zhang, V. S. Sheng, T. Li, and X. Wu, "Improving crowdsourced label quality using noise correction," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 29, no. 5, pp. 1675–1688, 2018.